МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования
«**Московский технический университет связи и информатики**»
(**МТУСИ**)

Кафедра «Системное программирование»

ОТЧЕТ
по лабораторной работе №14
Вариант №26

по дисциплине «**Параллельные и распределенные вычисления**»

Выполнил:
студент гр. БФИ2202
_____ Сидорук Д. В.
« ____ » _____ 2026 г.

Проверил:
старший преподаватель
_____ Клешнин Н. Г.
ассистент
_____ Соловьев А. С.
« ____ » _____ 2026 г.

Москва, 2026 г.

Содержание

1 Введение 3
2 Цель работы 3
3 Задание 3
4 Ход работы 3
Заключение 10

1 Введение

В данной лабораторной работе исследуется и реализуется алгоритм параллельного вычисления префиксной суммы (scan) на графическом процессоре с использованием технологии CUDA. Префиксная сумма является одной из фундаментальных операций параллельных вычислений и широко применяется в задачах обработки данных, компьютерной графики, сортировки, анализа массивов и других областях.

Целью работы является разработка эффективной реализации префиксной суммы для одномерного массива с использованием параллелизма GPU. В рамках работы необходимо реализовать ядро CUDA, обеспечивающее корректное и производительное вычисление префиксной суммы для входных данных произвольного размера (в пределах ограничений задачи).

Особое внимание уделяется оптимизации доступа к памяти, в частности использованию разделяемой памяти, а также корректной обработке граничных условий. В ходе выполнения работы также анализируются вычислительная сложность алгоритма, количество операций и обращений к памяти, а также возможности дальнейшей оптимизации.

2 Цель работы

Освоение освоение методов выполнения инклюзивного сканирования на GPU. Получение навыков расчета префиксных сумм одномерного списка с помощью CUDA.

3 Задание

Реализовать программу, выполняющую расчет префиксных сумм одномерного списка на GPU с помощью CUDA.

4 Ход работы

В листинге ниже приведено содержание файла rvs_laboratories_13/rvs_laboratories_13.cu, который содержит код, выполняющую расчет префиксных сумм одномерного списка на GPU с помощью CUDA (лист. 1).

Лист. 1 – Содержание rvs_laboratories_13/rvs_laboratories_13.cu

```
// На вход подается список (lst) длины n
// Результатом является префиксная сумма = {lst[0], lst[0] + lst[1],
// lst[0] + lst[1] + ... + lst[n-1]}

#include "wb.h"

#define BLOCK_SIZE 512 //@@ Это можно поменять

#define wbCheck(stmt) \
```

```
do {
  cudaError_t err = stmt;
  if (err != cudaSuccess) {
    wbLog(ERROR, "Failed to run stmt ", #stmt);
    wbLog(ERROR, "Got CUDA error ...  ", cudaGetErrorString(err));
    return -1;
  }
} while (0)

__global__ void addBlockSums(float *output, float *partialSums, int len) {
  int tID = threadIdx.x;
  int bID = blockIdx.x;
  int start = 2 * bID * blockDim.x;
  int offset = 2 * tID;

  float sum = (bID > 0) ? partialSums[bID - 1] : 0.0f;

  if (start + offset < len) {
    output[start + offset] += sum;
  }
  if (start + offset + 1 < len) {
    output[start + offset + 1] += sum;
  }
}

__global__ void scan(float *input, float *output, float *partialSums, int len) {
  extern __shared__ float temp[];

  int tID = threadIdx.x;
  int bID = blockIdx.x;
  int start = 2 * bID * blockDim.x;
  int offset = 2 * tID;

  temp[offset] = (start + offset < len) ? input[start + offset] : 0.0f;
  temp[offset + 1] = (start + offset + 1 < len) ? input[start + offset + 1] : 0.0f;

  __syncthreads();

  for (int d = 1; d <= blockDim.x; d *= 2) {
    int index = 2 * d * (tID + 1) - 1;
    if (index < 2 * blockDim.x) {
      temp[index] += temp[index - d];
    }
```

```
    __syncthreads();
  }

  if (partialSums != NULL) {
    if (tID == 0) {
      partialSums[bID] = temp[2 * blockDim.x - 1];
    }
  }

  if (tID == 0) {
    temp[2 * blockDim.x - 1] = 0.0f;
  }
  __syncthreads();

  for (int d = blockDim.x; d >= 1; d /= 2) {
    int index = 2 * d * (tID + 1) - 1;
    if (index < 2 * blockDim.x) {
      float t = temp[index - d];
      temp[index - d] = temp[index];
      temp[index] += t;
    }
    __syncthreads();
  }

  if (start + offset < len) {
    output[start + offset] = temp[offset] + input[start + offset];
  }
  if (start + offset + 1 < len) {
    output[start + offset + 1] = temp[offset + 1] + input[start +
     ↪ offset + 1];
  }
}

int main(int argc, char **argv) {
  wbArg_t args;
  float *hostInput;  // Входной одномерный список
  float *hostOutput; // Выходной список
  float *deviceInput;
  float *deviceOutput;
  float *devicePartialSums; // Массив для хранения сумм блоков
  int numElements; // количество элементов в списке

  args = wbArg_read(argc, argv);

  wbTime_start(Generic, "Importing data and creating memory on
   ↪ host");
  hostInput = (float *)wbImport(wbArg_getInputFile(args, 0),
   ↪ &numElements);
  hostOutput = (float *)malloc(numElements * sizeof(float));
  wbTime_stop(Generic, "Importing data and creating memory on
   ↪ host");

  wbLog(TRACE, "The number of input elements in the input is ",
        numElements);
```

```

  wbTime_start(GPU, "Allocating GPU memory.");
  wbCheck(cudaMalloc((void **)&deviceInput, numElements * sizeof(float)));
  wbCheck(cudaMalloc((void **)&deviceOutput, numElements * sizeof(float)));
  wbTime_stop(GPU, "Allocating GPU memory.");

  wbTime_start(GPU, "Clearing output memory.");
  wbCheck(cudaMemset(deviceOutput, 0, numElements * sizeof(float)));
  wbTime_stop(GPU, "Clearing output memory.");

  wbTime_start(GPU, "Copying input memory to the GPU.");
  wbCheck(cudaMemcpy(deviceInput, hostInput, numElements * sizeof(float),
                     cudaMemcpyHostToDevice));
  wbTime_stop(GPU, "Copying input memory to the GPU.");

  dim3 dimBlock(BLOCK_SIZE, 1);
  int numBlocks = (numElements + 2 * BLOCK_SIZE - 1) / (2 * BLOCK_SIZE);
  dim3 dimGrid(numBlocks, 1);
  size_t sharedMemSize = 2 * BLOCK_SIZE * sizeof(float);

  wbCheck(cudaMalloc((void **)&devicePartialSums, numBlocks * sizeof(float)));

  wbTime_start(Compute, "Performing CUDA computation");
  scan<<<dimGrid, dimBlock, sharedMemSize>>>(deviceInput, deviceOutput, devicePartialSums, numElements);

  if (numBlocks > 1) {
    scan<<<dim3(1,1), dimBlock, sharedMemSize>>>(devicePartialSums, devicePartialSums, NULL, numBlocks);
    addBlockSums<<<dimGrid, dimBlock>>>(deviceOutput, devicePartialSums, numElements);
  }

  cudaDeviceSynchronize();
  wbTime_stop(Compute, "Performing CUDA computation");

  wbTime_start(Copy, "Copying output memory to the CPU");
  wbCheck(cudaMemcpy(hostOutput, deviceOutput, numElements * sizeof(float),
                     cudaMemcpyDeviceToHost));
  wbTime_stop(Copy, "Copying output memory to the CPU");

  wbTime_start(GPU, "Freeing GPU Memory");
  cudaFree(deviceInput);
  cudaFree(deviceOutput);
  cudaFree(devicePartialSums);
  wbTime_stop(GPU, "Freeing GPU Memory");
```

```
146
147   wbSolution(args, hostOutput, numElements);
148
149   free(hostInput);
150   free(hostOutput);
151
152   return 0;
153 }
```

В листинге ниже приведено содержание файла dataset_generator/dataset_generator.cpp, который содержит код, генерирующий тестовые входные и контрольные выходные данные (лист. 2).

Лист. 2 – Содержание dataset_generator/dataset_generator.cpp

```
1  #include "gputk.h"
2
3  static char *base_dir;
4
5  static void compute(int *output, int *input, int num) {
6    int accum = 0;
7    int ii;
8    for (ii = 0; ii < num; ++ii) {
9      accum += input[ii];
10     output[ii] = accum;
11   }
12 }
13
14 static int *generate_data(int n) {
15   int *data = (int *)malloc(sizeof(int) * n);
16   int i;
17   for (i = 0; i < n; i++) {
18     data[i] = rand() % 4;
19   }
20   return data;
21 }
22
23 static void write_data(char *file_name, int *data, int num) {
24   int ii;
25   FILE *handle = fopen(file_name, "w");
26   fprintf(handle, "%d", num);
27   for (ii = 0; ii < num; ii++) {
28     fprintf(handle, "\n%d", *data++);
29   }
30   fflush(handle);
31   fclose(handle);
32 }
33
34 static void create_dataset(int datasetNum, int dim) {
35   const char *dir_name =
36       gpuTKDirectory_create(gpuTKPath_join(base_dir, datasetNum));
37
38   char *input_file_name  = gpuTKPath_join(dir_name, "input.raw");
```

```
  char *output_file_name = gpuTKPath_join(dir_name, "output.raw");

  int *input_data  = generate_data(dim);
  int *output_data = (int *)calloc(sizeof(int), dim);

  compute(output_data, input_data, dim);

  write_data(input_file_name, input_data, dim);
  write_data(output_file_name, output_data, dim);

  free(input_data);
  free(output_data);
}

int main() {
  base_dir = gpuTKPath_join(gpuTKDirectory_current(), "ListScan", "Dataset");
  create_dataset(0, 16);
  create_dataset(1, 64);
  create_dataset(2, 93);
  create_dataset(3, 112);
  create_dataset(4, 1120);
  create_dataset(5, 9921);
  create_dataset(6, 1233);
  create_dataset(7, 1033);
  create_dataset(8, 4098);
  create_dataset(9, 4018);
  return 0;
}
```

В листинге ниже приведено содержание файла CMakeLists.txt, определяющего правила сборки программы (лист. 3).

Лист. 3 – Содержание CMakeLists.txt

```
cmake_minimum_required(VERSION 3.27)
project(rvs_laboratories_13 LANGUAGES CXX)

option(BUILD_DATASET_GENERATOR "Build dataset generator" ON)
option(BUILD_MAIN_PROGRAM "Build main program" ON)

if (BUILD_DATASET_GENERATOR)
        set(LIBGPUTK_PATH "/usr/local/libgputk" CACHE STRING "Path to libgputk")

        add_executable(dataset_generator dataset_generator/dataset_generator.cpp)

        target_include_directories(dataset_generator PRIVATE ${LIBGPUTK_PATH}/include)
        target_link_directories(dataset_generator PRIVATE ${LIBGPUTK_PATH}/lib/)
        target_link_libraries(dataset_generator gputk)
```

```
endif()

if (BUILD_MAIN_PROGRAM)
        enable_language(CUDA)
        add_executable(rvs_laboratories_13
         ↪  rvs_laboratories_13/rvs_laboratories_13.cu)
endif()
```

Сгенерируем тестовые входные данные и проверим, что они действительно были сгенерированы (рис. 1).

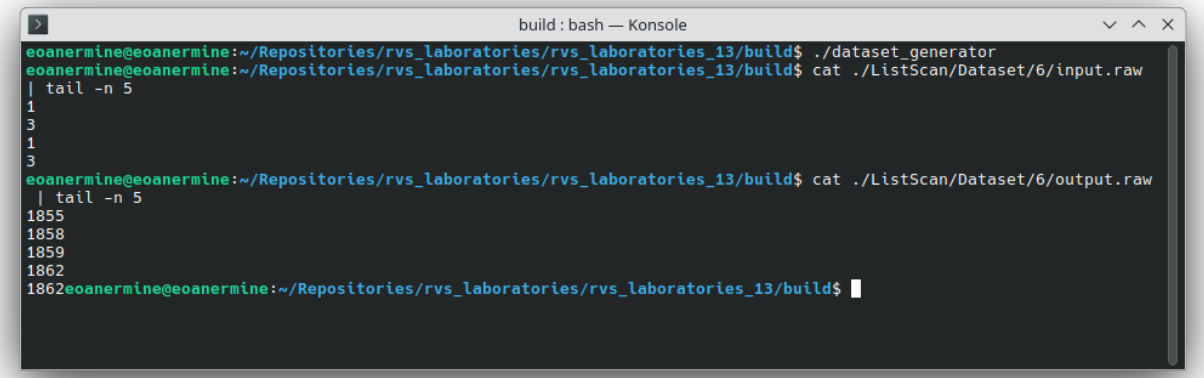

Рис. 1 – Генерация тестовых входных данных

На рисунке ниже приведен результат сборки и выполнения программы (лист. 2)

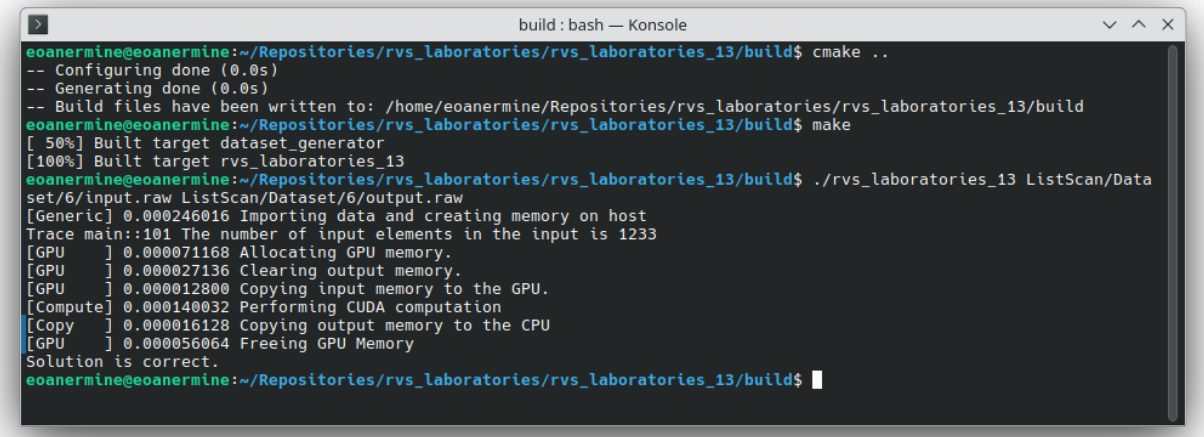

Рис. 2 – Компиляция и выполнение программы

Как можно видеть, вычисление префиксных сумм было выполнено корректно и успешно.

Заключение

Таким образом, в ходе выполнения данной лабораторной работы были освоены методы выполнения инклюзивного сканирования на GPU и получены навыки расчета префиксных сумм одномерного списка с использованием CUDA. На практике была реализована программа, выполняющая расчет префиксных сумм одномерного списка на GPU с помощью CUDA.